

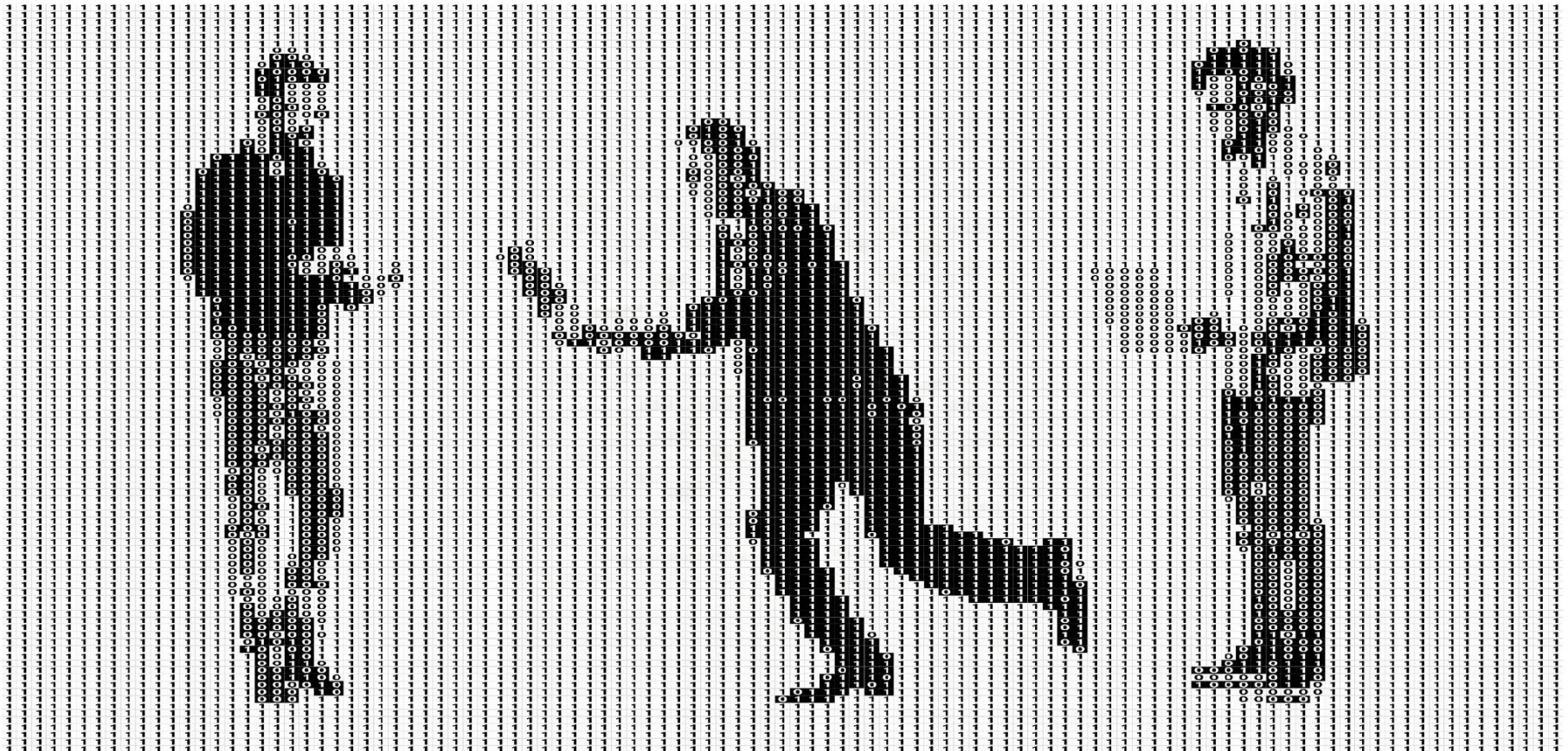


ILLUSTRATION: RASMUS SCHÖNNING

KODNING ■ Softwareudviklere, programmører og dataingeniører står til at vinde mest på AI. Hvis de ikke bliver fyret først.

Uventede vindere

LASSE HJORTH MADSEN

Data scientist og journalist

KRONIK

Den største AI-triumf ses uden sammenligning inden for softwareudvikling. Hvis man skriver kode som en del af arbejdet, skal man være ualmindeligt koldblodig for ikke at ryste en lille smule i bukserne over jobsikkerheden. Da jeg selv installerede Claude Code for godt et år siden, overtog den praktisk talt fra den ene dag til den anden den sjoveste del af mit arbejde: at skrive smart kode, der på elegant vis løser svære problemer. Det er ikke ret meget kode, jeg selv har skrevet siden da, og det kan stadigvæk give gåsehud at se, hvor meget bedre (og hurtigere!) Claude er.

Claude Code er et af de foretrukne hjælpeværktøjer til kodning og kommer fra firmaet Anthropic. Værktøjet giver mod betaling adgang til sprogmodeller som Fable 5 – modellen, der en kort overgang blev forbudt, fordi den var for god til at hacke sig gennem sikkerhedssystemer.

Man skal med andre ord være optimistisk grænsende til det naive for ikke at kunne forestille sig en snarlig jobmassakre på alle, som lever af at skrive computerkode.

Men der er også håb. Efter at have arbejdet intensivt med data, kodning og de nye AI-værktøjer er det tydeligt for mig, at nogle faktorer trækker i en anden retning. Folk, der koder, høster nemlig også større fordele af sprogmodellerne end alle andre. Så hvis vi undgår at blive fyret, ligger vi lunt i svinget.

Der er to væsentlige grunde til, at kodetyperne kan blive AI-vindere.

SAMMENSÆT ET TEAM

For det første kan man meget, meget mere med store sprogmodeller, hvis man fletter dem ind i lidt kode frem for at chatte løs i den velkendte brugergrænseflade på chatgpt.com, eller hvor ens foretrukne model nu lever. Det er ikke kun, fordi værktøjer som Claude Code er mere aktive, altså at de rent faktisk piller ved filer og installerer og afvikler programmer på computeren. Mange almindelige brugere har også fundet vej til værktøjer som Claude Cowork, der gør noget af det samme.

De egentlige superkræfter kommer, når man kombinerer den stringente logik fra traditionel computerkode med de mere uforudsigelige output fra store sprogmodeller. For eksempel ved at få modellerne til at arbejde sammen på snedig vis.

Her er en almindelig situation, som illustrerer ideen: Lad os sige, at jeg har en eller anden tung opgave. Måske handler det om at sammenfatte videnskabelige artikler eller indviklede love i et kort og letforståeligt sprog. Hvilken model skal jeg bruge til formålet? Hvilken AI-model leverer den bedste kvalitet til prisen?

På min arbejdsplads, en medicinalvirksomhed, har jeg i øjeblikket over 90 forskellige store sprogmodeller til rådighed. Den dyreste, men ikke nødvendigvis den bedste, koster mere end tusind gange så meget at bruge som den billigste. Hvilken én skal jeg vælge? Vi ved det ikke, og det ene gæt kan være lige så godt som det andet.

Én mulighed er at lade en af de nyeste og mest hypede sprogmodeller tage lederrollen og uddelegere opgaven til en eller flere passende modeller under hensyntagen til pris og evner. En anden kraftig model kan passende få opgaven som dommer og vurdere, hvilken af de konkurrerende løsninger der er bedst.

Pludselig har man sat et hold af modeller op i et samarbejde, der tilsammen kan være langt mere effektivt end en hvilken som helst enkelt model. Præcist som et velfungerende team med leder, kvalitetskontrol og udførende kræfter. Et godt team er næsten altid bedre end selv den dygtigste ener. Metoden kaldes orkestrering, fordi hver model har en klart afgrænset rolle, ligesom medlemmerne af et orkester.

Orkestrering kræver bare, at man tilgår sprogmodellerne med computerkode og klart definerer spillereglerne: Hvem er dirigent, og hvem bedømmer musikken. Eftersom det hele er bagt ind i almindelig kode, kan man også nemt køre processen igennem mange gange og se, hvorvidt resultatet er stabilt, og om kvaliteten er i orden.

Teknikken giver samtidig mulighed for såkaldte guardrails, en slags virtuelt autoværn, som også bare er almindelig kode, der sikrer, at produktet overholder visse spilleregler. Hvis vi for eksempel ikke tolererer brugen af emojis i sammenfatning af lovtekster, kan man lave noget håndfast logik, der støvsuger sammenfatningen for den slags uønskede tegn.



Pointen er, at de sætter en i forvejen imponerende teknologi på turbo.

Pointen er ikke, at teknikker som orkestrering fjerner de mange problemer, der kan være med brugen af AI. Pointen er, at de sætter en i forvejen imponerende teknologi på turbo. Det kræver blot, at man kan og vil tilsætte en lille smule computerkode til sit AI-eventyr. Det er en mulighed, der ligger til højrebænet, for folk, der koder.

VÆR PINAGTIGT SPECIFIK

Den anden AI-fordel, kodefolket har frem for alle andre, handler om den generelle tilgang: Hvor mange behandler deres chatbot som en blanding af orakel, samtalepartner og avanceret søgemaskine, ser udviklerne som udgangspunkt modellerne som det, de er, nemlig (store) matematiske funktioner. De undgår virkelig mange faldgruber ved at være ultraspecifikke, både med input (den berømte prompt) og med krav til outputtet.

For nylig rådgav jeg et team af kvalitetskontrollører i en stor virksomhed. De ville gerne konfigurere en skræddersyet chatbot til at hjælpe med den kedelige del af arbejdet. Men de havde en tendens til at forvirle sig ud i lange »samtaler« med deres chatbot – samtaler, som måske, måske ikke førte til en løst opgave. Og når opgaven blev løst, sad de uvægerligt tilbage med en forståelig usikkerhed omkring kvaliteten af løsningen – og hvorvidt de egentlig havde sparet tid.

En softwareudvikler ville instinktivt have været meget mere klar og formuleret instrukser i stil med: 1) Her er to dokumenter. 2) Kontrollér, om der er overensstemmelse på netop denne måde. 3) Aflever konklusionen i præcis dette format. Præcis, som når man skriver en funktion i et hvilket som helst computersprog: Man er tvunget til at være pinagtigt specifik om både forventninger til input og output.

Ansatte, der er uvante med den måde at tænke på, kan godt ende i et kaninhul, der starter med en prompt cirka sådan her: »Jeg foretager kvalitetskontrol på disse to dokumenter. Hjælp mig.«

Så ligesom alle andre teknologier har AI potentiale til at skubbe ret kraftigt til magtbalancen på arbejdsmarkedet, for hverken risikoen eller gevinsten er jævnt fordelt.

Softwareudviklere, data scientists, dataingeniører og lignende typer har et damoklessværd med bogstaverne AI indgraveret hængende over hovedet. Men de har også meget bedre muligheder end alle andre for at bruge sværdet. Det hele afhænger af, om de når at gribe det, før det falder.